

Ans: to the question no-1

①

User story 1:

"As a user, I want to login in securely so that I can access my account."

Tasks:

1. Design login page UI (To Do)
2. Set up secure backend authentication mechanism (e.g., JWT or OAuth) (To Do)
3. Implement input validation for email/password (To Do)
4. Integrate secure communication protocols (e.g., HTTPS) (To Do)
5. Create unit tests to prevent vulnerabilities (e.g., SQL injection, XSS) (To Do)

User Story 2:

"As a user, I want to search for products by category to find items easily."

Tasks:

1. Design search bar UI with category dropdown (To Do)
2. Build database schema for products category (To Do)
3. Develop search API to fetch products by category (To Do)

4. Integrate API with the frontend search functionality (To Do)

5. Add error handling for no results found (To Do)

6. Perform usability tests for the search feature (To Do)

(11)

Prioritization During Sprint Planning

1) Value to the Customer:

a) User Story-1: (Login Securely) is a high-priority feature since users cannot access the application without secure login.

b) User Story-2: (Search for Products) provides utility but is secondary compared to user authentication.

2) Technical Feasibility:

a) Tasks related to User Story 1 involve foundational work (e.g., backend authentication HTTPS setup) and are likely prerequisites for other features.

b) Tasks for User Story 2 require search API development, which can be complex but not critical for the first Sprint.

3) Sprint Planning Decision:

a) Prioritize all tasks of User Story 1 in the current Sprint.

b) Begin User Story 2 by tackling simpler tasks (e.g., UI design and database schema setup).

c) Defer more complex tasks of User Story 2 to a subsequent Sprint.

Tracking Tasks Using a Scrum Board columns:

- To Do: All tasks not yet started.
- In Progress: Tasks being actively worked on by the team.
- Done: Completed tasks verified by QA or team review.

Example Scrum Board:

Tasks	To Do	In Progress	Done
Design login page UI	✓		
Set up secure backend authentication	✓		
Implement input validation	✓		
Integrate secure communication	✓		
Create unit tests for login	✓		
Conduct security tests	✓		
Design search bar UI	✓		
Build database schema	✓		

How Tasks Progress During the Sprint

- Team members move tasks from To Do to In Progress when starting work.
- After completing a task, it moves to the Done column once verified.
- At the end of the Sprint, completed tasks for User Story 1 ensure it is fully delivered.

while partially completed User Story 2 tasks can be reprioritized.

Ans: to the Q No-2

Methodologies and Their Approach to Risk Management and Adaptability

i) Spiral Methodology

• Risk Management:

- a) The Spiral model explicitly focuses on risk assessment at each phase.
- b) Each iteration (or "spiral") begins with identifying and addressing the most significant risks, ensuring high-risk components are tackled early.
- c) Prototyping and stakeholder feedback at every iteration reduce uncertainties and clarify client needs.

• Adaptability:

- a) Allows incremental development and client feedback at each iteration, making it possible to adjust requirements and plans.
- b) However, the structured approach can be slower compared to Agile methods and may be less

responsive to rapidly changing requirements.

ii) Agile Methodology

• Risk Management:

- a) Risk is mitigated by delivering working software in small, incremental sprints, allowing the team to address issues early.
- b) Continuous feedback ensure that potential risks, such as misaligned client expectations are identified and corrected quickly.

• Adaptability:

- a) Highly flexible, Agile thrives in environments with evolving requirements.
- b. Agile's iterative nature and focus on collaboration make it easy to adapt to changes while keeping development aligned with client needs.

iii) Extreme Programming (XP)

• Risk Management:

- a) XP reduces technical risks through practices like Test-Driven Development (TDD), pair.

Programming, and continuous integration.

b) Frequent releases minimize the risk of building the wrong product by ensuring ongoing alignment with client expectations.

• Adaptability:

a) XP's emphasis on constant feedback and close collaboration with the client allows for rapid adaptation to new or changing requirements.

b) Practices like refactoring ensure the codebase remains maintainable and adaptable as the product evolves.

Most Suitable Methodology

Recommendation: Agile Methodology

Reasons:

a) Evolving requirements:

- Agile is specifically designed to handle projects with uncertain and changing requirements. Its iterative nature ensure that the software evolves alongside the client's needs.

b) Risk Management:

- By delivering incremental value in each sprint Agile minimizes the impact of high-risk components and ensure continuous alignment with client expectations.

c) Flexibility and Collaboration:

- Agile allows for seamless collaboration between the development team and the client which is crucial for refining the product based on feedback and reducing uncertainty.

Comparison:

- While the Spiral Model excels in risk management, its slower, structured approach may not suit rapidly evolving requirements.
- Extreme Programming (XP) is highly adaptable, but it requires an exceptionally disciplined team and may not address high-level risks as comprehensively as Agile.

Agile strikes the right balance between risk

management and adaptability, making it the most effective choice for this project.

Ans: to the question no-3

Comparison of Methodologies

Aspect	Waterfall	Agile	Extreme Programming (XP)	Spinal
Adaptability	Low	High	Very High	Moderate
Predictability	High	Moderate	Low	Moderate
Risk Management	Moderate	High	High	Very High
Best For	Fixed requirements	Evolving needs	Frequent releases	High-risk projects

Methodology Recommendations for Projects

Project A:

- Well-defined requirements and strict deadline.
- Recommended Methodology: Waterfall Model.

Reasoning:

- Predictability: The linear structure and well-defined phases ensure clear timelines and deliverables, making it easier to meet the strict deadline.
- Stable Requirements: With no significant change expected, the rigidity of the Waterfall model is not a disadvantage.
- Risk Management: Since requirements are fixed, the project's risks can be addressed upfront during the planning phase.

Project B:

- Evolving requirements, uncertain timeline, and continuous feedback.
- Recommended Methodology: Agile Methodology.

Reasoning:

- Adaptability: Agile thrives in environments where requirements are dynamic customer feedback is frequent.

- Customer Collaboration: Agile fosters continuous communication with the client, ensuring the product evolves according to their needs.
- Risk Management: Incremental delivery allows risks to be identified and mitigated early in the development process.

Ans: to the question no - 4

Principles of Software Engineering Ethics

- Public Interest: Prioritize safety, privacy and welfare.
- Client and Employer: Serve clients/employers while protecting public interest.
- Product Quality: Ensure high-quality, reliable, and secure software.
- Professional Integrity: Maintain honesty and avoid conflicts of interest.
- Fairness: Avoid bias and treat everyone fairly.

Issues in Professional Responsibility

- Accountability: Responsibility for software outcomes.

- Confidentiality: Protect sensitive information.
- Bias: Avoid discriminatory designs.

Role of ACM/IEEE code of Ethics

- Public Good: Focus on societal benefit and harm avoidance.
- Competence: Work within expertise to ensure quality.
- Transparency: Communicate risks openly.
- Conflict Resolution: Address ethical dilemmas collaboratively and ethically.

The code guides ethical decision-making to ensure professional standards and societal values are upheld.

Ans: to the question no-5

Functional Requirements:

a) User Registration and Login:

- Enables secure access for passengers and staff.
- Contributes to security by restricting unauthorized access.

b) Flight Search and Booking:

- Allows users to search and book flights based on preferences.
- Enhances usability by providing easy access to available flights.

c) Payment Gateway Integration:

- Facilitates secure online payments for tickets.
- Supports performance by ensuring smooth transaction processing.

d) Real-time Flight Updates:

- Provides users with updates on delays, cancellations, or gate changes.
- Improves user experience by keeping users informed.

2) Customer Support System:

- Offers chat or call support for resolving issues.
- Enhances usability by addressing user concern promptly.

Non-Functional Requirements

a) System Availability:

- Ensures 24/7 access to the system.
- Contributes to performance and user trust.

b) Scalability:

- Handles a large number of simultaneous users during peak hours.
- Supports system maintenance and adaptability.

c) Response Time:

- Processes user actions (e.g., search or payment) within 2 seconds.
- Enhances usability by ensuring smooth interactions.

d) Data Security:

- Encrypts sensitive information like personal details and payment data.

- Strengthens security and builds user confidence

e) Maintainability:

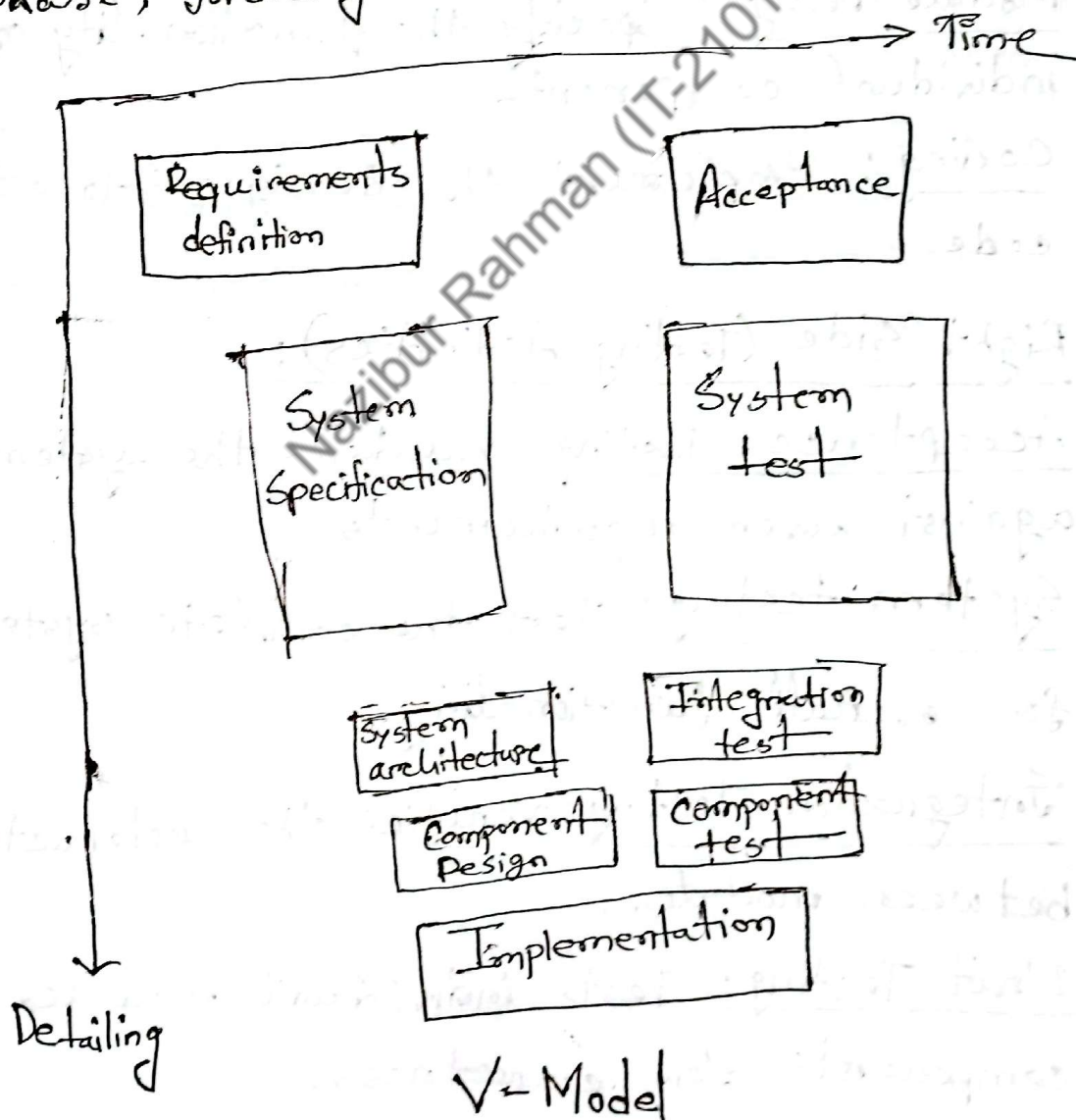
- Allows easy updates for adding new features or fixing bugs.
- Supports system maintenance and reduces downtime.

By addressing these functional and non-functional requirements, the system ensures a secure, efficient, and user-friendly experience while remaining robust and adaptable for future needs.

Ans: to the Question no-6

The V-Model of Testing Phases

The V-Model (Verification and Validation model) is a plan-driven approach where development and testing activities are closely aligned. It emphasizes the relationship between each development phase and its corresponding testing phase, forming a "V" shape.



Explanation

a) Left Side (Development Activities):

- Requirements: Define the system's functionality and user expectations.
- System Design: Break down requirements into detailed specifications.
- Architectural Design: Define the system structure and module interactions.
- Module Design: Specify the functionality of individual components.
- Coding: Implement the designs into actual code.

b) Right Side (Testing Activities):

- Acceptance Testing: Validates the system against user requirements.
- System Testing: Test the complete system for overall functionality.
- Integration Testing: Verifies the interactions between modules.
- Unit Testing: Tests individual modules or components for correctness.

Relationships

- Each testing activities on the right corresponds directly to a development activity on the left.
- For example:

i) Requirements $\leftrightarrow$ Acceptance Testing: Ensures the final product meets the initial requirements.

ii) Module Design $\leftrightarrow$ Unit Testing: Verifies the correctness of individual modules as per their design.

The V-Model ensures that testing is planned early and conducted ~~methodology~~ methodically, reducing defects and aligning with the overall project goals.

Ans: to the question no- 7

Prototype Development Process

- i) Requirement Identification: Gather initial essential requirements.
- ii) Prototype Design and Development: Build a basic version with core functionality.
- iii) User Evaluation and Feedback: Present the prototype to users for feedback.
- iv) Refinement and Iteration: Modify the prototype based on feedback and refine it.
- v) Final System Development: Develop the complete system based on refined requirements.

Benefits of Prototyping

- i) User feedback: Continuous input ensures the system meets user needs.
 - ii) Risk Reduction: Identifies and resolves issues early, preventing costly revisions.
 - iii) Iterative Development: Allows for ongoing refinement, adapting to evolving requirements.
- Prototyping helps clarify requirements early, reduces

risks, and promotes iterative, user-driven development.

Ans: to the question no-8

Process Improvement Cycle in Software Engineering

- i) Planning: Define improvement goals and areas needing enhancement.
- ii) Measuring: Collect data on current process performance.
- iii) Analyzing: Identify inefficiencies or issues based on the data.
- iv) Improving: Implement changes to optimize the process.
- v) Controlling: Monitor the improvements to ensure long-term effectiveness.

Commonly Used Process Metrics

- i) Defect Density: Measures defects per unit of software, size, helping monitor code quality.
- ii) Cycle Time: Tracks time taken for tasks, helping identify delays or inefficiencies.
- iii) Code Churn: Measures code changes, indicating stability or rework needs.

iv) Customer Satisfaction: Assesses user satisfaction, ensuring products meets needs.

v) Velocity: Measures work completed per iteration, helping track team productivity.

These metrics help monitor, evaluate, and continuously improve software processes.

Ans: to the question no-9

SEI Capability Maturity Model (CMM)

The Capability Maturity Model (CMM) provides a framework for assessing and improving software development processes. It is structured across five levels, where each focuses on enhancing organizational processes and capabilities.

i) Level-I : Initial

- Characteristics: Processes are informal, unpredictable, and depend on individual effort.
- Contribution: This level highlights the need for establishing structured processes to improve the predictability of software delivery.

b) Level-II : Managed

- Characteristics: Basic project management processes are introduced, such as tracking cost, schedule, and performance.
- Contribution: The introduction of formal management processes helps reduce project risks and makes software delivery more predictable.

c) Level-III : Defined

- Characteristics: Standardized, documented processes are implemented across the organization, ensuring consistency and integration.
- Contribution: Defined processes allow for greater efficiency and consistency, improving quality and reducing errors in the software development lifecycle.

d) Level-IV : Quantitatively Managed

- Characteristics: Processes are measured and controlled using metrics, allowing for better management of performance and quality.
- Contribution: The use of data-driven decisions enhances process stability and helps identify areas for improvement based on quantitative insights.

e) Level-v: Optimizing

- Characteristics: Focus is on continuous processes improvement through innovation, learning, and adapting based on performance data.
- Contribution: The organization continuously refines its processes, fostering a culture of improvement that leads to highly optimized, efficient development practices.

Ans: to the question no-10

Core Principles of Agile Software Development

a) Customer Collaboration over Contract Negotiation

- Emphasizes working closely with customers to meet their evolving needs.

b) Responding to Change over Following a Plan

- Agile values flexibility and adapting to change during development.

c) Individuals and Interactions over Processes and Tools

- Focuses on communication and collaboration among team members.

d) Working software Over Comprehensive Documentation

- Prioritizes delivering functional software over extensive documentation.

Application in Different Environments

- Small Projects: Agile works well due to flexibility and iterative delivery.
- Large Projects: Scaled Agile frameworks (e.g., SAFe) can be applied but face challenges with coordination.
- Startups: Agile is ideal for quickly iterating and adapting products based on customer feedback.
- Regulated Environments: Requires balancing flexibility with compliance requirements.

Benefits of Agile

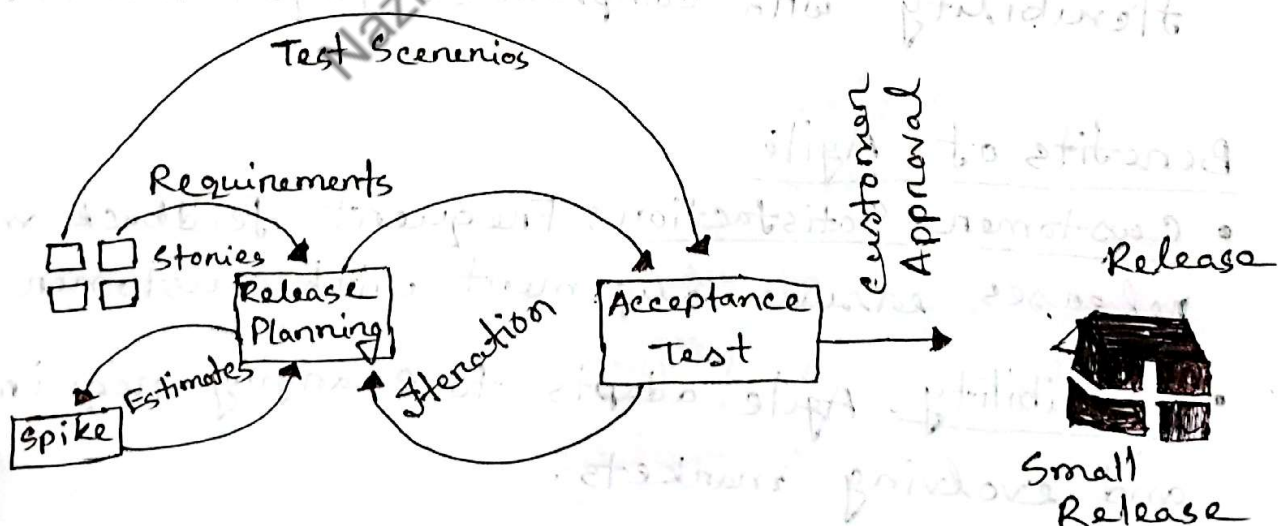
- Customer Satisfaction: Frequent feedback and releases ensure alignment with customer needs.
- Flexibility: Agile adapts to changing requirements and evolving markets.
- Faster Delivery: Short iterations lead to quicker releases and faster time to market.

Challenges of Agile

- Scope creep: Ongoing changes can lead to uncontrolled scope expansion.
- Coordination: Large teams may struggle with dependencies and integration.
- Documentations: Less emphasis on documentation can be problematic for regulated industries.

Ans: to the question no-11

Release cycle of Extreme Programming (XP) is given below:



Extreme Programming (XP) emphasizes flexibility, collaboration, and rapid iterations. Key practices include:

- a) Pair Programming: Two developers work together improving code quality and sharing knowledge.
- b) Test-Driven Development (TDD): ~~Write~~ Writing tests before code to ensure correctness and reduce bugs.
- c) Continuous Integration: Frequently integrating code to avoid issues and run automated tests and reduce bugs and run tests.
- d) Collective code ownership: All developers are responsible for the entire codebase.
- e) Coding Standards: Consistent coding practices to ensure clarity and maintainability.
- f) Small Release: Delivering software in frequent small updates for early feedback.
- g) Sustainable Pace: Working at a manageable pace to avoid burnout and ensure long-term productivity.

Ans: to the question no -12

To design an Entity-relationship Diagram (ERD) for the digital library management system, we'll breakdown the entities, their attributes and the relationships between them based on the requirements provided. Here's how it can be structured:

a) Entities and attributes:

Book Attributes:

- Book ID (Primary key): Unique identifier for each book.
- Title: Title of the book.
- Author: Author of the book.
- ISBN: International Standard Book Number.
- Genre: Genre of the book.

Member Attributes:

- Member ID (Primary key): Unique identifier for each member.
- Name: Name of the member.
- Contact Details: contact information (phone number, email etc.)
- Address: Physical address of the member.

Borrowing Activity

Attributes:

- Borrowing ID (Primary key): Unique identifier for each borrowing activity.
- Book ID (Foreign key): Refers to the borrowed book (links to the Book entity)
- Member ID (Foreign key): Refers to the member who borrowed the book (links to the Member entity).
- Borrowing Date: The date the book was borrowed.
- Return Due Date: The date the book is due for return.
- Fine: The fine imposed if the book is returned late (can be Null if returned on time).

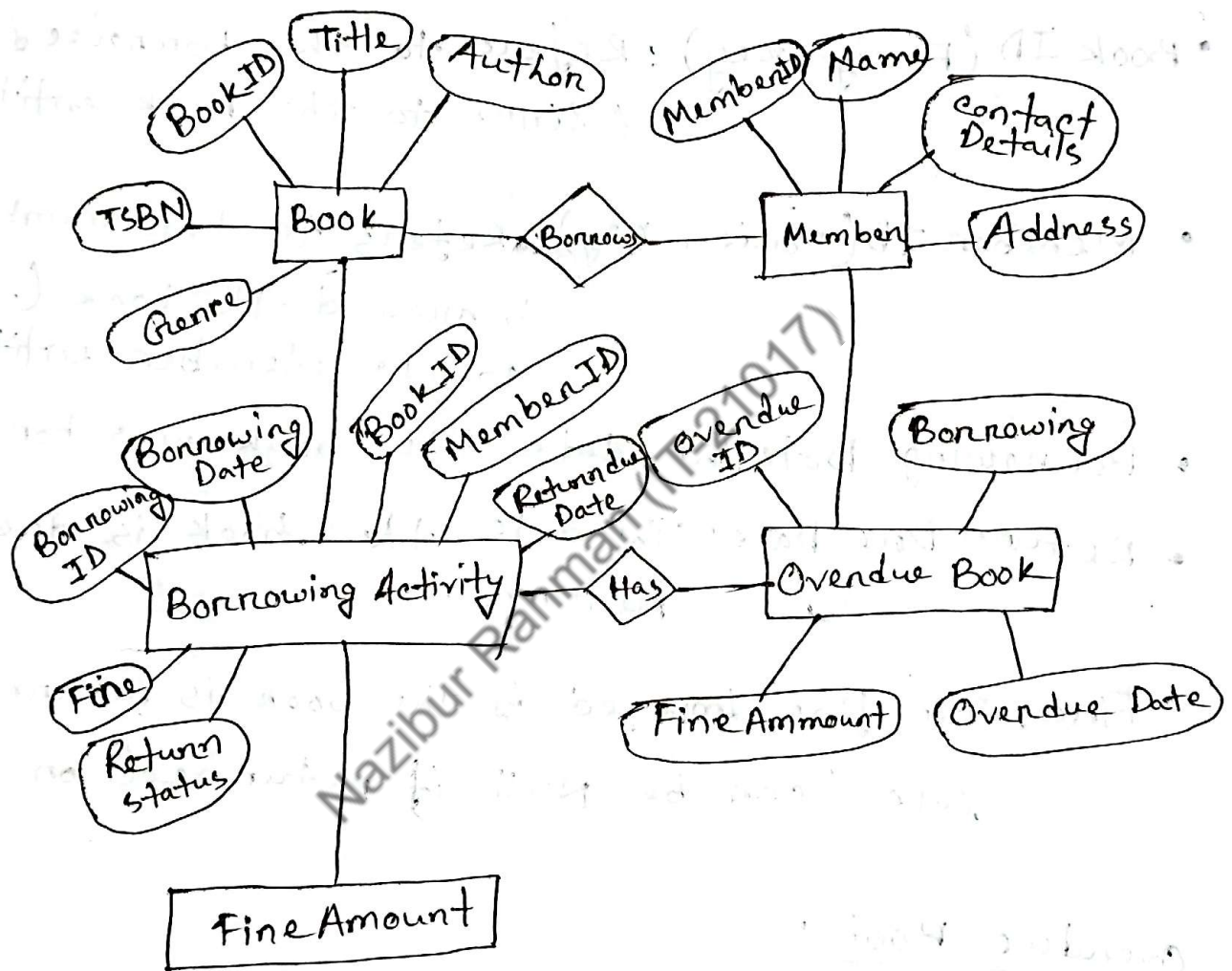
Overdue Book:

Attributes:

- Overdue ID (Primary key): Unique identifier for each overdue book entry.
- Borrowing ID (Foreign key): Refers to the borrowing activity of the overdue book (links to the Borrowing Activity entity).

- FineAmount: Fine imposed on the overdue book.
- OverdueDate: Date when book became overdue.

ERD Diagram:



b) Relationships:

Book- Borrowing Activity (One-to-Many)

- One book can be borrowed many times, but each borrowing activity refers to a specific book.
- Relationship: A book can appear in many borrowings.

activities, but each borrowing activity involves only one book.

Member-Borrowing Activity (One-to-Many)

- One member can borrow multiple books, but each borrowing activity refers to only one member.
- Relationship: A member can have many borrowing records, but each borrowing activity involves only one member.

Borrowing Activity - Overdue Book (One-to-One)

- A borrowing activity can either be on time or overdue. If overdue, an entry is made in the Overdue Book entity.

Ans: to the question no -13

What is Testing?

Testing is the process of evaluation a system on its components to identify errors, ensure quality, and verify that it meets specified requirements. It involves executing software to detect and ensure that the product function correctly.

Difference between validation and verification:

Aspect	Verification	Validation
Definition	Ensures that the Product is being built correctly.	Ensures that the right product is being built.
Objective	Checks whether the Software conforms to design, requirements and standards.	Checks whether the Software fulfills its intended purpose.
Process Type	Static Process.	Dynamic Process.
Example	Code reviews, design Verification.	Functional and user acceptance testing.

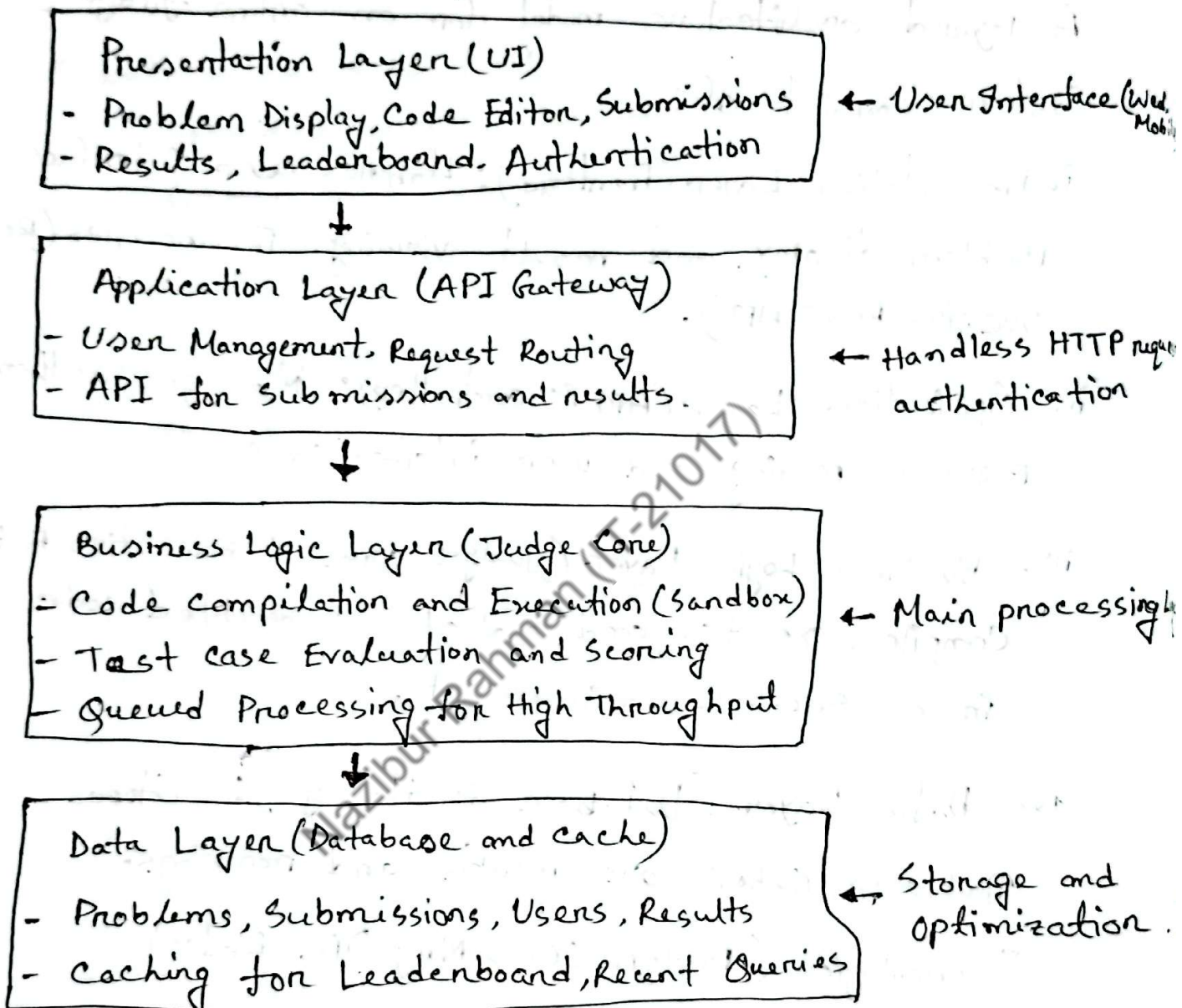
Ans: to the question no-14

A layered architecture model for an online judge

System consist of:

- i. Presentation Layer (Frontend): Handles user interface, Problem display, and result viewing. Technologies (React, Vue.js, REST API).
- ii. Application Layer (API & controller): Manages authentication, request routing, and user interactions.
- iii. Business Logic Layer (Judge core & Execution Engine): Compiles, executes and evaluates submitted code in a secure environment.
- iv. Data Layer (Database & Storage): Stores Problems, Submissions, results and rankings. Technologies (PostgreSQL, MongoDB, Redis)

Layered Architecture Model for an Online Judge System:



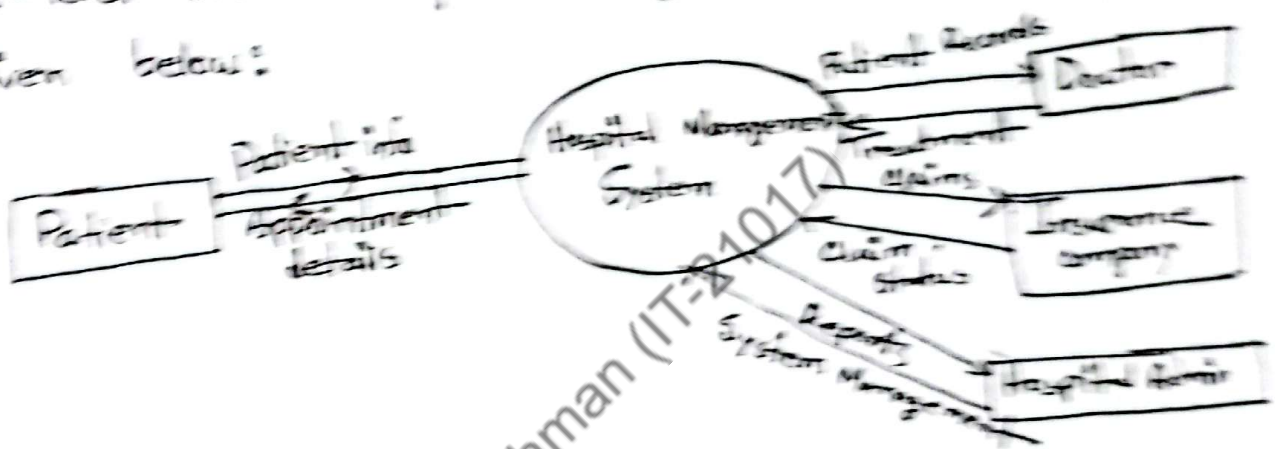
Ensuring Scalability, Maintainability, and Performance

- Scalability: Load balancing, parallel execution and
- Maintainability: Modular design, microservices, and clear separation of concerns.
- Performance: Asynchronous processing, optimized database queries, and distributed execution.

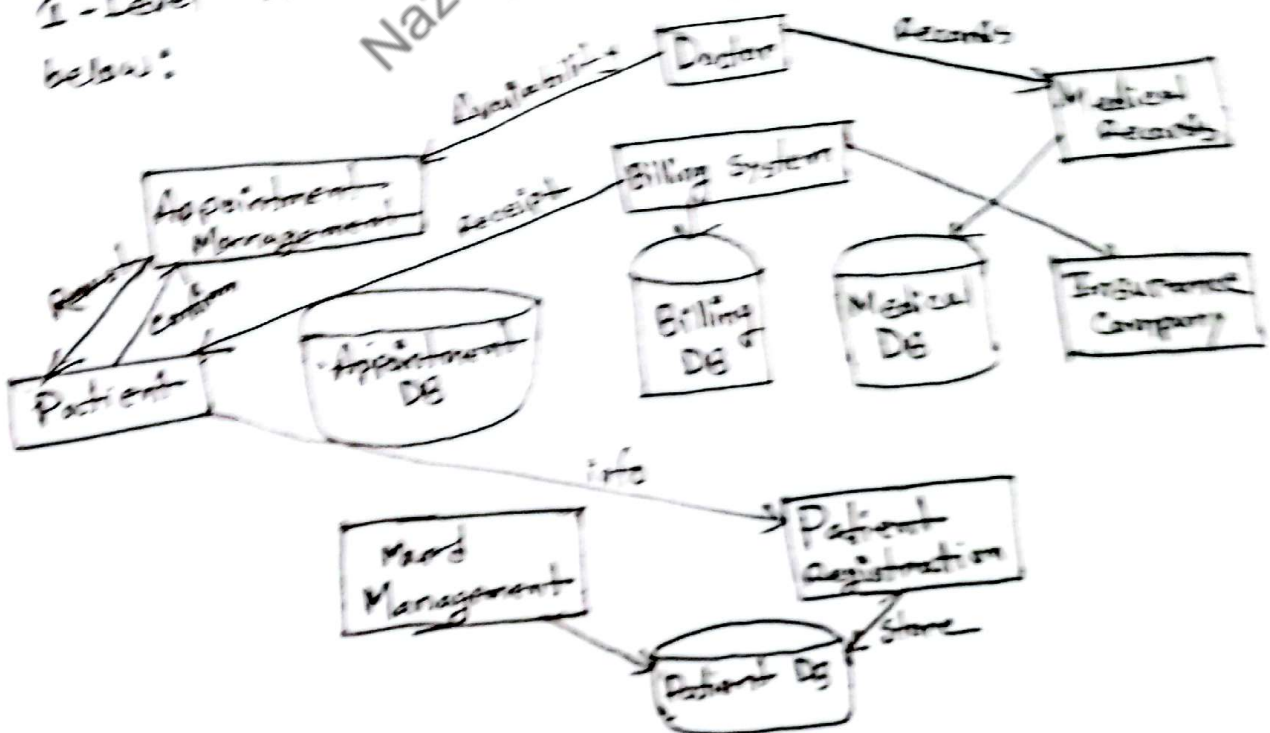
This architecture ensures efficient and reliable online judging, even under high traffic.

Ans to the question no-15

C-level DFD of Hospital Management System is given below:



1-Level DFD of Hospital Management System is given below:



Ans: to the question no - 16

To design a UML Class Diagram for the scenario depicted by the provided Sequence Diagram we need to identify the key classes, their attributes, methods and relationships. Here's the breakdown:

Scenario Description

- i) A Student interacts with a Login Screen to Log in.
- ii) The Login Screen validates the user by interacting with a Database.
- iii) If the login is successful, the Login Screen displays the classList (grades) to the Student.

UML Class Diagram

Classes and Their Attributes / Methods

a) Student

- Attributes:

- i) studentID: String

- ii) name: String

- Methods:

- i) login (Username: String, password: String): boolean

b) LoginScreen:

- Attributes:

- i) username: String
- ii) password: String

- Methods:

- i) validateUser (username: String, password: String): boolean
- ii) displayClassList (grades: List<String>): void

c) Database:

- Attributes:

- i) userData: Map<String, String> (stores username and password)
- ii) classListData: Map<String, List<String>> (stores grades for each user)

- Methods:

- i) validateUser (username: String, password: String): boolean
- ii) getClassList (username: String): List<String>

d) Class List

- Attributes:

- i) grades: List<String>
- ii)

- Methods:

- i) getGrades(): List<String>

Relationships

- i) Student interacts with LoginScreen to log in.
 - Relationship: Association (Student $\rightarrow$ LoginScreen)
- ii) LoginScreen interacts with Database to validate the user and fetch the class list.
 - Relationship: Association (LoginScreen $\rightarrow$ Database)
- iii) LoginScreen displays the classList to the Student.
 - Relationship: Association (LoginScreen $\rightarrow$ classList)

Nazibur Rahman (IT-21017)

Ans: to the question no-17

Quality Assurance (QA) vs. Quality Control (QC).

Quality Assurance (QA): It is process-oriented and proactive, focussing on preventing defects by improving processes and methodologies throughout the SDLC.

Quality Control (QC): It is product-oriented and reactive, concentrating on identifying and fixing defects in the final product through testing and inspections.

Difference between QA and QC:

Aspect	Quality Assurance (QA)	Quality Control (QC)
Focus	Focuses on process improvement and preventing defects.	Focuses on identifying and fixing defects in the product.
Orientation	Process-oriented (how the product is made).	Product-oriented (what the final product is).
Approach	Proactive, aiming to improve processes for consistent quality.	Reactive, focusing on detecting and fixing defects.
Activities	Process models, checklists, guidelines and methodologies.	Test case, bug reports, test execution, and reviews.

Tools Used	Process models, checklist guidelines, and methodology	Test case, bug reports, test execution, and reviews.
Timing	Done throughout the SDLC from planning to maintenance.	Primarily done during or after development and testing

Impediments to QA and QC:

Impediments to QA:

- i) Lack of clear processes: Without well-defined processes and standards, QA efforts can be inconsistent and ineffective.
- ii) Poor communication: If QA teams are not in sync with developers, misunderstandings can arise affecting process improvement efforts.
- iii) Lack of management support: Without strong support from management, it can be challenging to implement and maintain effective QA processes.
- iv) Resistance to change: Team members may resist adopting new processes or improvements, slowing down the implementation of QA best practices.
- v) Inadequate training: If the team is not adequately trained in process management, QA efforts can fall short.

Impediments to QC:

i) Insufficient test coverage: Inadequate test coverage may result in undetected defects, leading to poor product quality.

ii) Limited resources: Limited time or manpower can hinder effective testing, reducing the ability to catch defects.

iii) Unclear specifications: If the product requirements are not clear, QC teams may face challenges in identifying what to test for, leading to incomplete testing.

iv) Time pressure: Tight deadlines may lead to rushing through QC processes, which can result in defects being overlooked.

v) Defect complexity: Complex defects may be hard to identify with traditional testing methods, especially when the product is large or intricate.

Ans: to the question no-18

No, Quality Assurance (QA) is not just about finding bugs early, but also about ensuring that the software meets the required quality standards at each stage of development, optimizing the entire development process.

Role of QA in SDLC:

- i. Requirement Gathering & Analysis: QA ensures that requirements are clear, complete and testable, identifying any ambiguities or inconsistencies early.
- ii. Design: QA reviews design documents to ensure they meet requirements, are testable, and address usability, security, etc.
- iii. Implementation: QA prepares test cases, performs unit tests, and ensures adherence to coding standards and specifications.

iv. Testing: QA conducts various types of testing to identify defects and works with developers to fix them.

Ans: to the question no-19

The RAD model is a software development methodology that emphasizes rapid prototyping, quick iterations and active user participation to deliver software solutions quickly and efficiently.

Key Phases:

- i. Requirements planning
- ii. User Design
- iii. Construction
- iv. cutover

Principles:

- i. Prototyping: Users interact with working prototypes, providing feedback that drives system improvements.

- ii. Iterative Development: Software is built in iterations, allowing for frequent adjustments and refinements.
- iii. Time boxing: Development is done within fixed, short time frames to ensure rapid delivery.

Advantages:

- Faster Delivery: Prototyping and parallel development allow quicker delivery of functional software.
- User Satisfaction: Active involvement ensures the final product matches user expectations.
- Flexibility: Changes can be incorporated easily as user needs evolve.

How RAD Supports Faster Delivery and Quality:

- Faster Delivery: Prototypes and iterative cycles reduce development time by delivering working software quickly.
- Quality maintenance: Constant user feedback and testing catch issues early, ensuring the final product is robust and meets user needs.

Ans: to the question no-20

Test cases table:

Decision	x input	y input	Expected output
if (y==0)	5	0	"y is zero"
else if (x==0)	0	3	"x is zero"
Loop does not run	0	2	No output
Numbers divisible	4	2	"2", "4"
Numbers not div.	4	3	"3"
Edge case (y=0)	5	-2	"2", "4"
Edge case (x=0)	-3	2	No output

Java unit testing:

JUnit test class

```
import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;
import java.util.ArrayList;
import java.util.List;
```

```
Public class DecisionTest {
```

```
    Private List <String> output;
```

```
@ Before
```

```
    Public void setUp() {
```

```
        output = new ArrayList<>();
```

```
    }
```

```
    Private void process(int n, int y) {
```

```
        if (y == 0) {
```

```
            Println("y is zero");
```

```
        }
```

```
        else if (n == 0) {
```

```
            Println("x is zero");
```

```
        }
```

```
        else {
```

```
            for (int i = 1; i <= n; i++) {
```

```
                if (i * y == 0) {
```

```
                    Println(String.valueOf(i));
```

```
                }
```

```
            }
```

```
        }
```

```
@ Test
```

```
    public void testYisZero() {
```

```
        output.clear();
```

```
        process(5, 0);
```

```
        assertEquals(List.of("y is zero"), output);
```

```
    }
```

@Test

public void testXisZero() {

output.clear();

Process(0, 3);

assertEquals(List.of("x is zero"), output);

@Test

public void testLoopDoesNotRun() {

output.clear();

Process(0, 2);

assertEquals(List.of(), output);

Ans: to the question no-21

To implement exception handling, setup, and timeout rules in JUnit 4:

I. Setup Function: Use `@Before` to set up resources before each test, or `@BeforeClass` for setup before any tests.

Example:

`@Before`

```
public void setup() { ... }
```

II. Exception Handling: Use `@Test(expected = Exception.class)` to expect an exception or use the `ExpectedException` rule for more control.

Example:

```
@Test(expected = IllegalArgumentException.class)
```

```
public void testException() { ... }
```

III. Timeout Rule: Use timeout rule to set a time limit for a test. If the test exceeds the limit, it fails.

Example:

`@Rule`

```
public Timeout globalTimeout = Timeout.seconds(1);
```

Full Example:

```
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Rule;
import org.junit.Test;
import org.junit.rules.ExpectedException;
import org.junit.rules.Timeout;
```

```
public class MyTest {
```

```
    @BeforeClass
```

```
    public static void setUpBeforeClass() {
```

```
        System.out.println("Setting up resources before  
all tests...");
```

```
    }
```

```
    @Before
```

```
    public void setUp() {
```

```
        System.out.println("Setting up resources before  
each test...");
```

```
    @Rule
```

```
    public ExpectedException exceptionRule =
```

```
        ExpectedException.none();
```

```
    @Rule
```

```
    public Timeout globalTimeout = Timeout.seconds(1);
```

@Test

```
public void testException() {  
    exceptionRule.expect(IllegalArgumentException.class);  
    exceptionRule.expectMessage("Test exception");  
    throw new IllegalArgumentException("Test exception");  
}
```

@Test

```
public void testTimeout() throws InterruptedException  
{  
    Thread.sleep(1500);  
}
```

Nazibur Rahman (IT-21017)